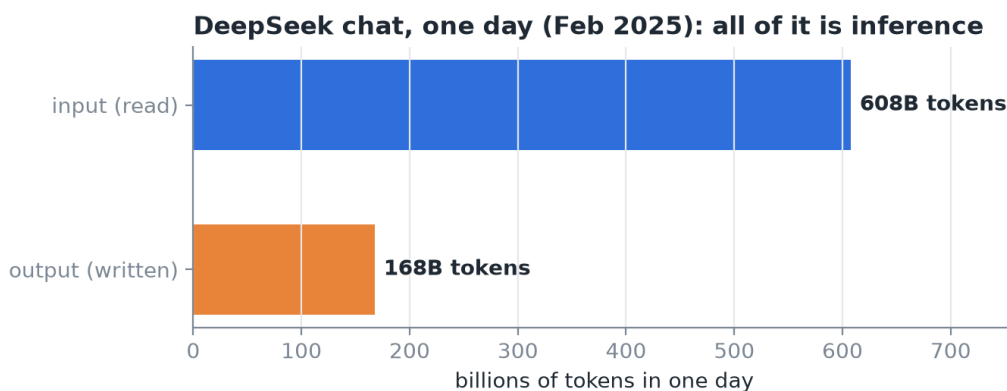


01 · What Is Inference & Inference Engineering?

Source: [fanout.sh / inference-eng / in-01](https://fanout.sh/inference-eng/in-01) · ~9 min · Next: *The Inference Stack*

TL;DR. Inference is running a trained model token by token, every time someone uses it, so across a fleet it usually costs more than training. Decode is limited by memory bandwidth (moving weights), not arithmetic. Latency, throughput and cost pull against each other, and batching is the first lever. The job is a loop: measure, find the bottleneck, change one thing, measure again.

1. Why inference matters: the DeepSeek bill



$\approx 1,800 \text{ GPUs} \times 24 \text{ h} \times \$2/\text{GPU-h} \approx \$87\text{k}$ for the day

DeepSeek chat service, 24 h, Feb 2025: ~1,800 GPUs on average, ~\$87k for the day at \$2/GPU-hour.

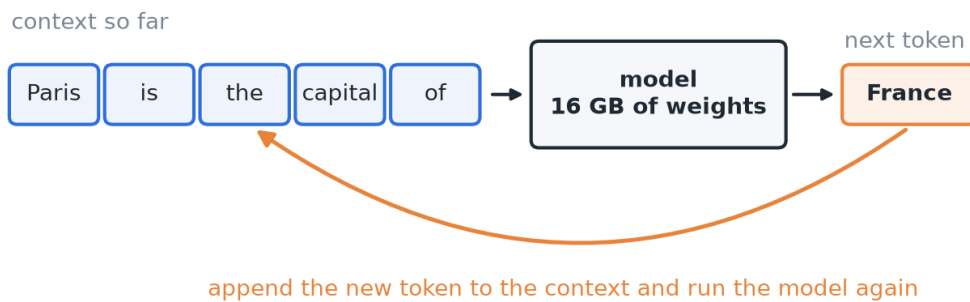
- One day of DeepSeek chat: **608B input tokens** read, **168B output tokens** written.
- Ran on **~1,800 GPUs** on average. At \$2/GPU-hour: $1,800 \times 24 \times \$2 \approx$ **\$87k/day**.
- The model was trained months earlier. Every one of those tokens was inference.

Key point: the gap between serving well and serving badly comes from a small number of engineering decisions. That gap is the job.

2. What inference actually is

- A trained model is **just a file of numbers**. Llama 3 8B has about 8B params at 16 bits (2 bytes each), so the file is **about 16 GB**.
- **Inference = running it**. Feed in text, predict the next token, append it, and repeat until done. This is **autoregressive** generation.

Autoregressive generation: one token per model run



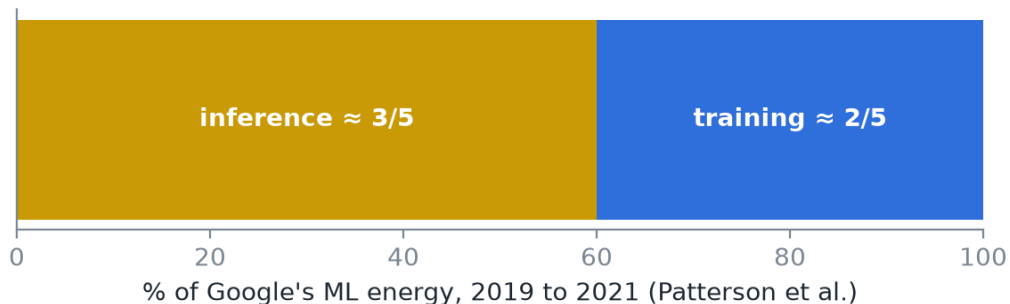
The autoregressive loop: each generated token is fed back in as input.

Training vs inference

	Training	Inference
How often	Once, or a few times	Every request, every chat, every agent step
Cost shape	Huge one-off	Ongoing, scales with usage

- Google fleet-wide, 2019 to 2021: **~3/5 of ML energy went to inference** and ~2/5 to training.
- **"Train once, serve forever."** This lopsided cost is why inference is its own discipline.

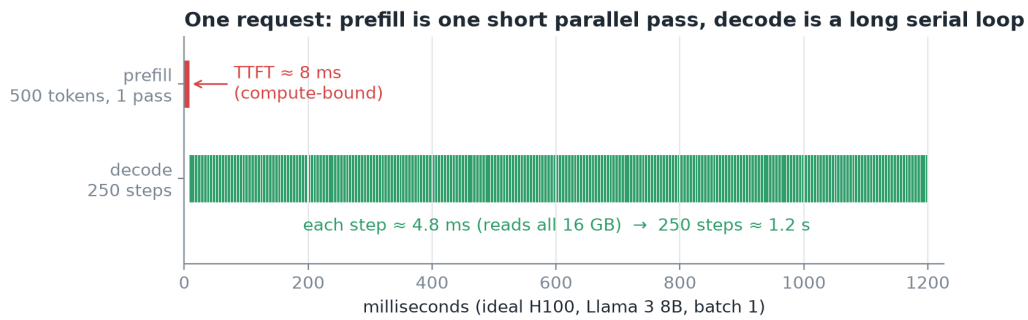
Train once, serve forever



Google ML energy use, 2019 to 2021 (Patterson et al.).

3. One request: prefill and decode

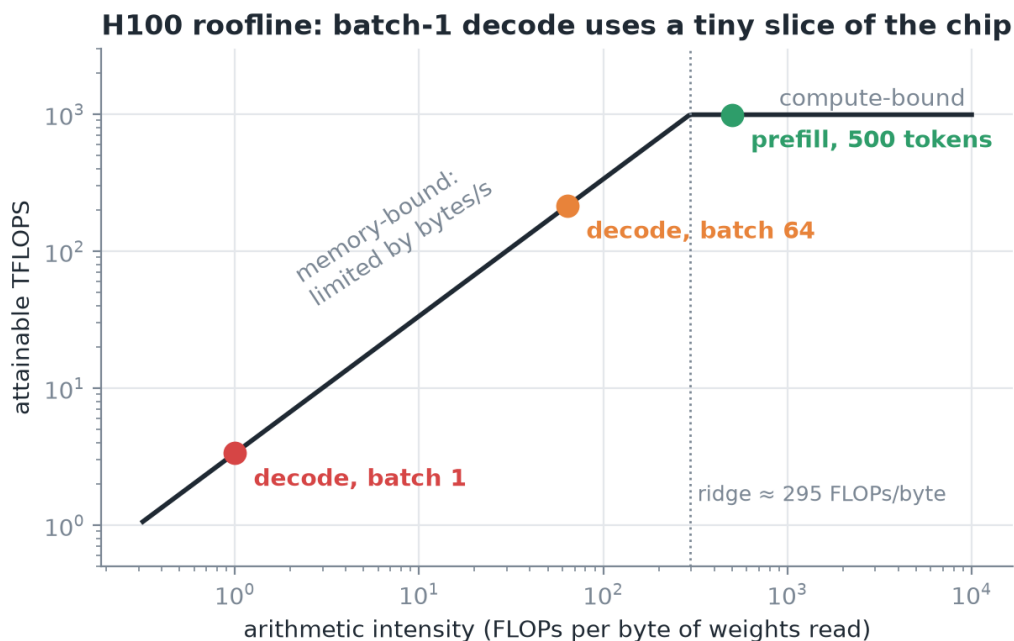
Running example: a chat app serving **Llama 3 8B on one H100**. The prompt is ~500 tokens and the answer is ~250 tokens.



Ideal timings: prefill handles all 500 prompt tokens in one pass (the red sliver is time to first token), then decode runs 250 serial steps of about 4.8 ms each.

- **Prefill:** all 500 prompt tokens go through the model in **one parallel pass**. This causes the pause before the first word, called **TTFT (time to first token)**.
- **Decode:** generate one token, append, run again. **250 sequential steps**.
- **The surprise:** every decode step reads **all 16 GB of weights** from GPU memory (HBM) to produce **one token**.

The bandwidth ceiling (memorize this calculation)



*Batch-1 decode does about 1 FLOP per byte of weights read, far left of the ridge, so memory bandwidth sets its speed.
Prefill sits past the ridge and is compute-bound.*

H100 HBM bandwidth ≈ 3.35 TB/s
 Weights per step = 16 GB
 Time per step = $16 / 3350$ s ≈ 4.8 ms
 Max tokens/s (1 user) $\approx 1 / 4.8$ ms ≈ 200 tok/s

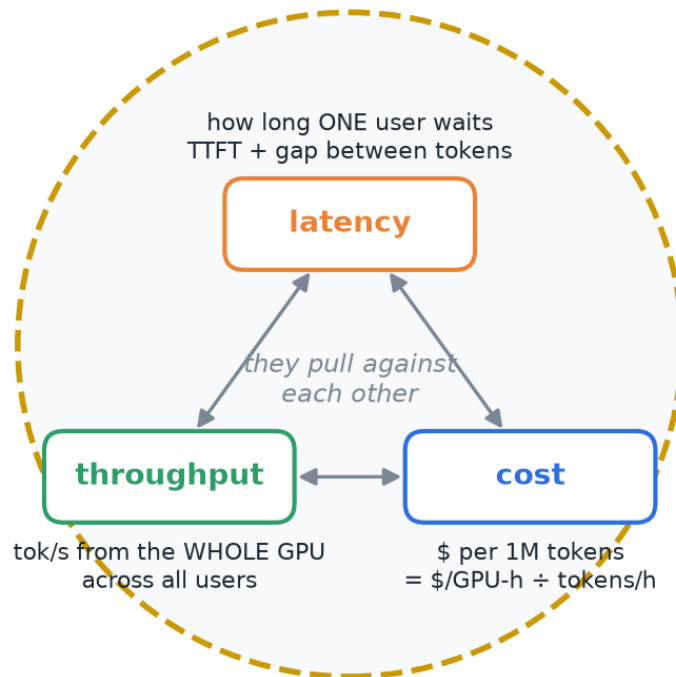
This is a hard ceiling, even with perfect code.

- The chip could do far more math. The limit is **how fast bytes arrive**. Decode at batch 1 is **memory-bandwidth-bound**.

4. The three numbers (and one constraint)

The three numbers an inference engineer watches

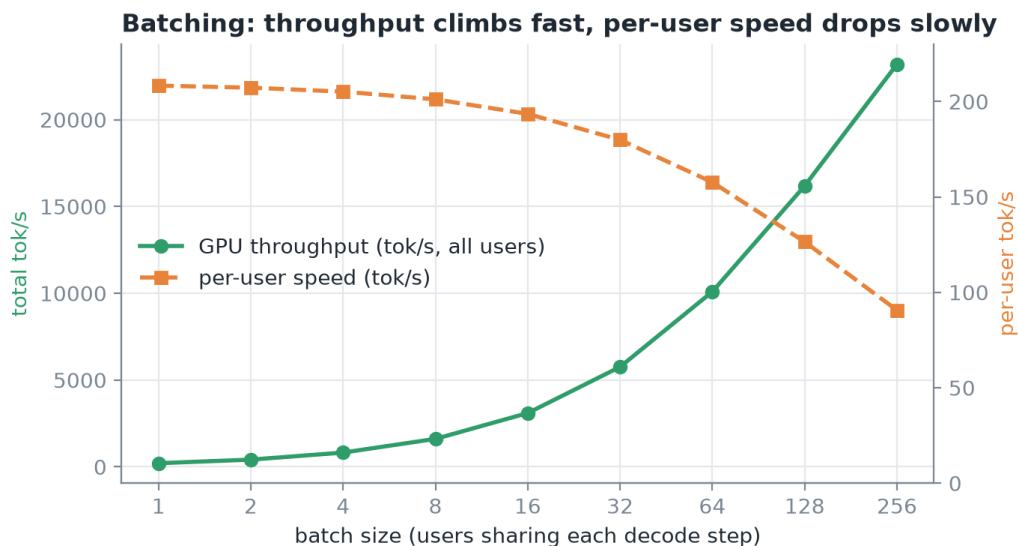
quality: a hard constraint around all three



Quality isn't traded: a faster but worse answer is not a win.

- The gap between tokens is called **ITL** (inter-token latency) or **TPOT** (time per output token).

The first trade-off: batching



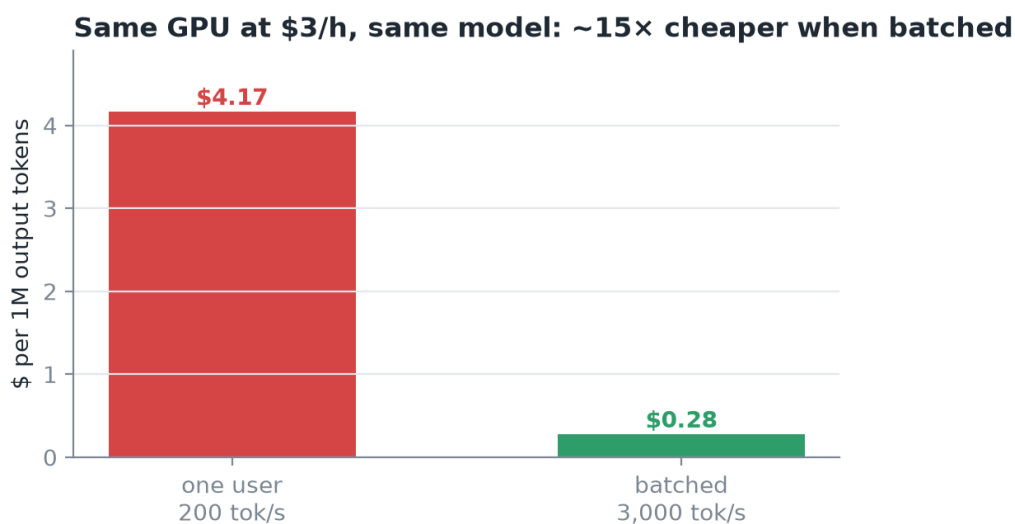
Idealized upper bound: bandwidth model with Llama 3 8B KV cache (128 KB/token). Real engines land lower.

One read of the weights serves every user in the batch, so total throughput climbs steeply while each user's speed falls only gradually.

- **Batch 1:** read 16 GB and get 1 token.

- **Batch 64:** read the same 16 GB and get **64 tokens**, one per user. The weight read is **amortized**.
- Throughput jumps. Each step does more work, so **per-user latency goes up a bit**.
- **"The whole game is a better deal on this trade."** Much of the course is about getting more throughput for less added latency.

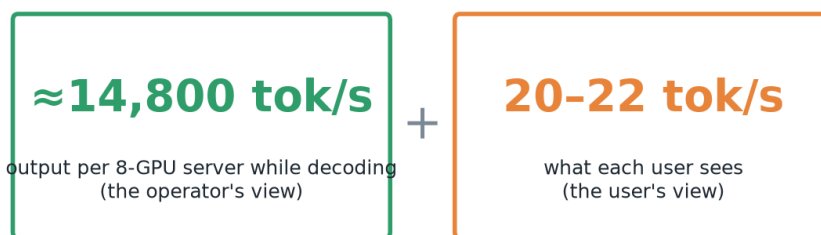
5. Putting money on it



Illustrative prices, real arithmetic. Cost per 1M tokens = $\$3 \div (\text{tok/s} \times 3,600) \times 10^6$.

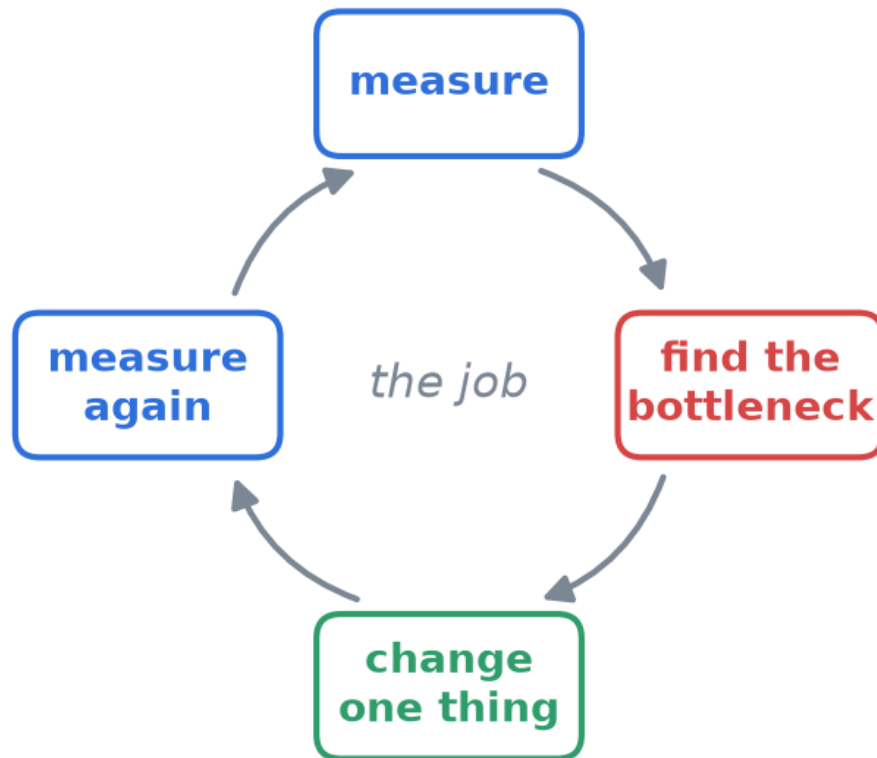
Real-world evidence: - **vLLM (2023)**: up to **24×** the throughput of plain Hugging Face Transformers, with the same GPUs and the same weights.

DeepSeek V3/R1 in production: both numbers good at the same time



6. What an inference engineer does all day

The inference engineer's loop



Six lever areas, from the fleet down to the chip

closer to users

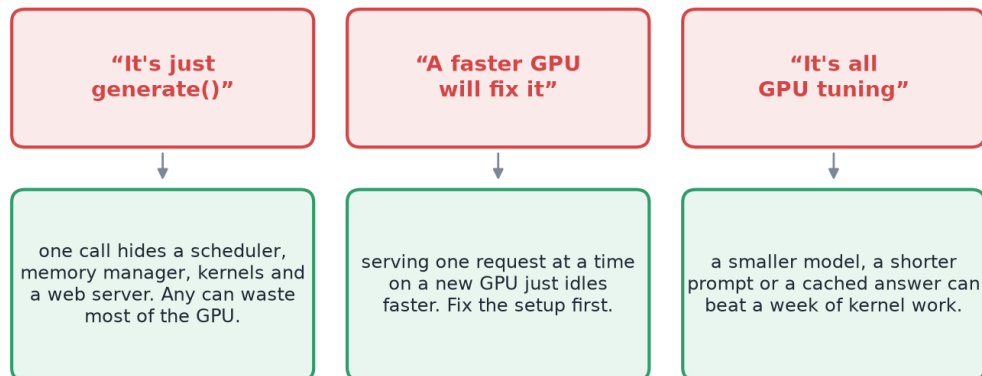
<i>fleet</i>	the fleet	route requests to the right replica; autoscale with traffic
<i>server</i>	scheduling	pick which requests share each step; no stalls
<i>model</i>	precision	8/4-bit weights: fewer bytes moved per step
<i>model</i>	parallelism	split a too-big model across several GPUs
<i>GPU memory</i>	memory	pack the KV cache tightly; reuse shared prompt prefixes
<i>GPU compute</i>	kernels	fuse ops so data makes fewer trips through memory

closer to silicon

Each lever area gets its own module later in the course.

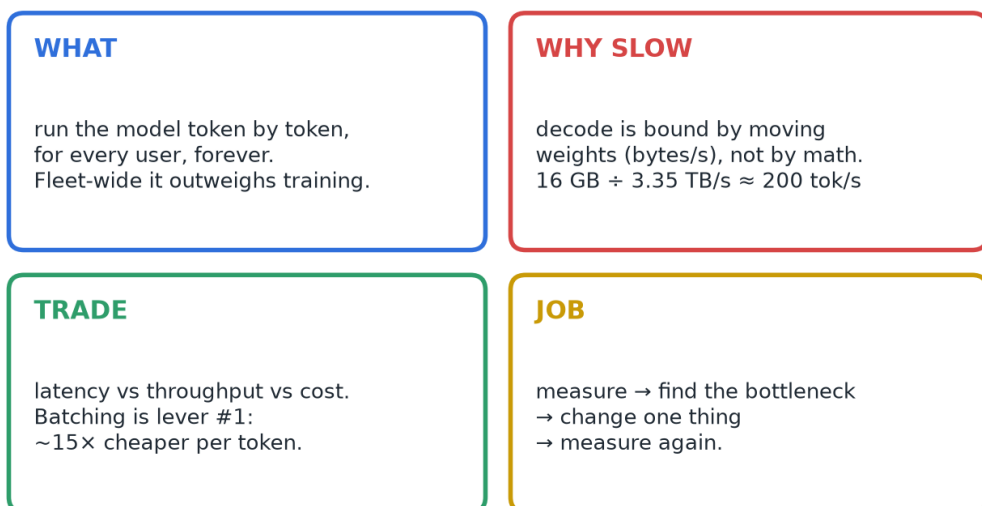
7. Three myths

Three myths (top) and what's actually true (bottom)



8. Recap

Lesson 01 on one card



Interview prep (my additions, beyond the video)

Formulas to have ready

- **Weight bytes** = params $\times$ bytes per param. $8\text{B} \times 2 \text{ B (FP16/BF16)} = 16 \text{ GB}$. INT8 gives 8 GB, INT4 gives about 4 GB.
- **Batch-1 decode ceiling** $\approx$ memory bandwidth $\div$ weight bytes. H100: $3.35 \text{ TB/s} \div 16 \text{ GB} \approx 200 \text{ tok/s}$.
- **Cost per 1M tokens** = (\$ per GPU-hour) $\div$ (tokens/s $\times$ 3,600) $\times 10^6$.
- **FLOPs per token (forward)** $\approx 2 \times$ params. For 8B, that is about 16 GFLOPs per token.
- **Arithmetic intensity at batch 1 decode**: $\sim 2 \text{ FLOPs per param} \div 2 \text{ bytes per param} \approx 1 \text{ FLOP/byte}$. H100 BF16 dense is $\sim 989 \text{ TFLOPS}$ against 3.35 TB/s, so its ridge point is **$\sim 300 \text{ FLOPs/}$**

byte. Decode at batch 1 is far below the ridge (memory-bound). Batching multiplies the FLOPs per byte of weights read, which is why it helps.

- **Prefill is compute-bound:** $500 \text{ tokens} \times 16 \text{ GFLOPs} \approx 8 \text{ TFLOPs}$, which is roughly 8 ms at peak on an H100. All tokens share one weight read.

Why batching isn't free

- Every user in the batch has their own **KV cache**. For Llama 3 8B that is $32 \text{ layers} \times 8 \text{ KV heads} \times 128 \text{ dims} \times 2 \text{ (K and V)} \times 2 \text{ bytes} \approx \mathbf{128 \text{ KB per token}}$. A 750-token conversation is ~94 MB, so 64 users need ~6 GB. KV memory, not compute, often caps the batch size.
- As the batch grows, the KV cache reads grow too, so the step time rises. That is the latency cost of batching.

Likely questions

Why is decode memory-bound but prefill compute-bound?

Decode processes one new token per sequence per step but must stream all weights, so there are very few FLOPs per byte. Prefill processes hundreds of tokens against a single weight read, so FLOPs per byte are high.

Estimate the max single-user tokens/s for a 70B BF16 model on 8xH100 with tensor parallelism.

$140 \text{ GB of weights} \div (8 \times 3.35 \text{ TB/s} \approx 26.8 \text{ TB/s}) \approx 5.2 \text{ ms per step}$, so about 190 tok/s as a ceiling. Real numbers are lower because of communication overhead and KV reads.

Your chat service's cost per token is too high. Where do you start?

Measure first: GPU utilization, batch size over time, TTFT/ITL percentiles and the tokens/s actually achieved. Common culprits are a low effective batch (fix with continuous batching), KV memory limits (fix with paged KV, KV quantization or prefix caching), weights that are too big (fix with FP8/INT4) or long prompts. Also check the non-GPU fixes: a smaller model, prompt trimming, response caching.

What are TTFT and ITL, and what drives each?

TTFT (time to first token) is driven by queueing plus prefill, so it scales with prompt length. ITL (inter-token latency, also called TPOT) is the time per decode step, driven by the weight and KV bytes read per step and by batch size.